ADVANCED DATA STRUCTURES AND ANALYSIS

(ADSA)

SLA MICRO PROJECT – STAGE 2 REPORT

Frequency-Based Optimization of Library Search using Optimal Binary Search Tree (OBST)

Group Members

Roll No.	Name of Student
24	Abhishek Kanaujiya
26	Gaurav Kute
30	Aditya Panigrahi
35	Vinay Phanse

TABLE OF CONTENTS

1. Problem Statement

1.1 Background and Motivation

In modern libraries, a huge number of books are stored digitally. As the size of the database increases, the efficiency of searching becomes very important.

Traditional searching techniques like **Linear Search** take O(n) time, which becomes very slow for large datasets. Even **Binary Search Trees (BST)**, though better, do not guarantee optimal performance because their structure depends on insertion order and not on usage patterns.

In real-world library systems:

- Some books (like Data Structures, DBMS, OS) are searched frequently
- Some books are rarely accessed

However, a normal BST does not consider this difference in frequency. As a result:

- Frequently searched books may lie deeper in the tree
- This increases the average number of comparisons

To solve this issue, we use **Optimal Binary Search Tree (OBST)**, which arranges nodes based on frequency to minimize overall search cost.

1.2 Problem Definition

Given a collection of books where:

- Each book has a **Book ID (key)**
- Each book has a **search frequency**

We need to design a system that:

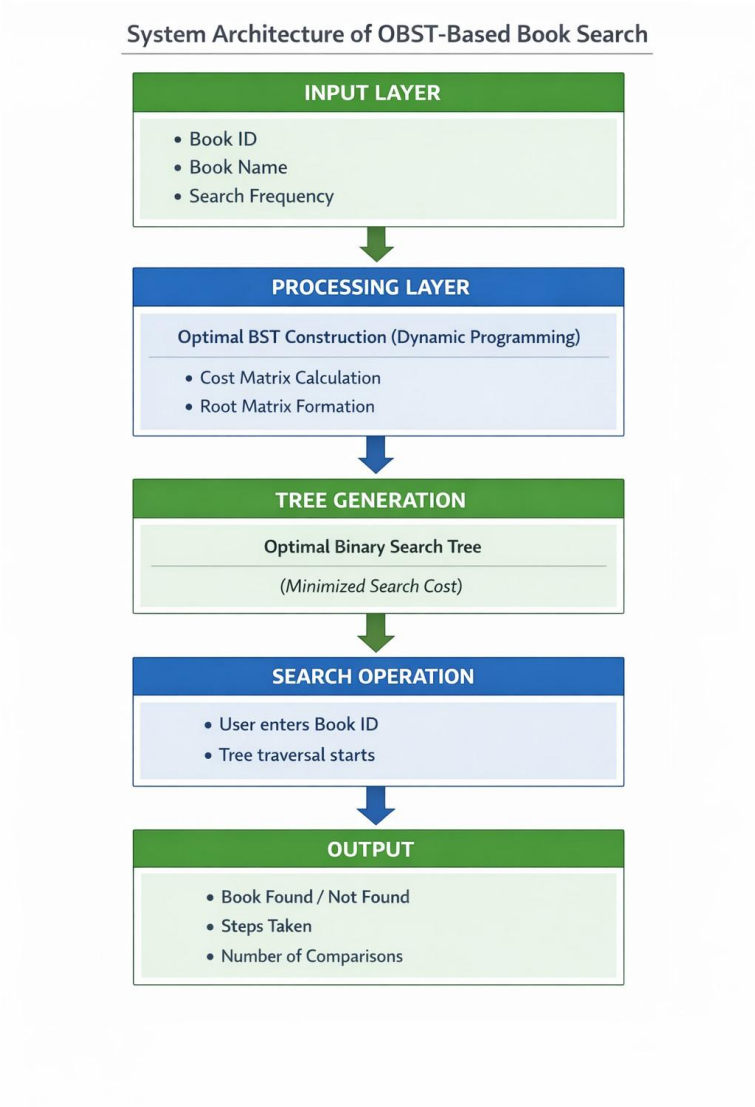
1. Organizes books using **Optimal Binary Search Tree**
2. Minimizes the **average search cost**
3. Improves search efficiency compared to normal BST
4. Displays the **step-by-step search process**
5. Counts the **number of comparisons** made during search

1.3 Objectives

The main objectives of this project are:

- To represent library books as nodes in a binary search tree
- To construct an **Optimal BST using frequency data**
- To reduce the average search time by placing frequently accessed books near the root
- To simulate search operations and track comparisons
- To compare the performance of OBST with traditional searching techniques

1.4 System Diagram:



2. Proposed Solution (Designed Methodology)

2.1 Graph Model Representation

The system represents books using two arrays:

- **Keys Array** → stores sorted Book IDs
- **Frequency Array** → stores how often each book is searched

Example:

Book IDs: 10, 20, 30, 40
Frequencies: 5, 2, 8, 3

This structured representation helps in building the OBST efficiently.

2.2 Dynamic Weight Computation

An OBST is a binary search tree designed in such a way that the **expected search cost is minimized**.

The idea is:

- Frequently accessed nodes should be placed near the root
- Less frequently accessed nodes can be deeper

The cost is calculated as:

Total Cost = Σ (frequency × depth)

Where:

- Depth = level of node in tree
- Root has depth = 1

Thus, OBST tries to minimize the weighted path length.

2.3 Modified Dijkstra Algorithm

OBST uses **Dynamic Programming** because:

- It solves overlapping subproblems
- It builds the solution using smaller subtrees

Steps involved:

1. Compute cost for all subtrees
2. Try every node as root
3. Choose the root that gives minimum cost
4. Store results in matrices to avoid recomputation

This ensures optimal structure of the tree.

2.4 Step-wise Algorithm (Pseudocode)

```
Algorithm: OBST(keys, freq, n)

OBST(keys, freq, n):

1. Initialize cost[i][i] = freq[i]

2. For length = 2 to n:
      For i = 0 to n-length:
          j = i + length - 1
          cost[i][j] = infinity

          For k = i to j:
              cost = cost[i][k-1] + cost[k+1][j] + sum(freq[i..j])

              If cost < cost[i][j]:
                  cost[i][j] = cost
                  root[i][j] = k

3. Construct tree using root matrix

4. Return OBST
```

2.5 Data Structures Used

Arrays

- Store book IDs and frequencies
- Provide quick access to elements

Cost Matrix

- Stores minimum cost for each subtree
- Helps in avoiding recomputation

Root Matrix

- Stores index of optimal root
- Used to construct the final tree

Binary Tree Structure

- Each node contains Book ID
- Left child contains smaller keys
- Right child contains larger keys

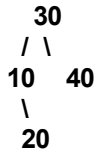
2.7 Solved Example

Input:

Keys: 10, 20, 30, 40

Frequencies: 5, 2, 8, 3

Final OBST :



Explanation:

- 30 is selected as root because it minimizes total cost
- Left subtree contains 10 and 20
- Right subtree contains 40

Search Example

Search Book ID: 20

Steps:

1. Compare with 30 → go left
2. Compare with 10 → go right
3. Compare with 20 → found

👉 Total Comparisons = 3

3. IMPLEMENTATION

3.1 Implementation Overview

The Library Management System using OBST is implemented as an interactive application that integrates data storage, algorithm processing, and user interaction.

The system follows a structured workflow:

- It first loads book data containing Book IDs, names, and their search frequencies.
- Then, it applies the **Optimal Binary Search Tree (OBST)** algorithm using dynamic programming.
- Based on the computed cost and root matrices, an optimized tree is constructed.
- The user can then perform search operations using Book ID.
- During the search, the system tracks and displays each comparison step.

The main aim of implementation is to simulate how OBST improves search efficiency in real-world library systems.

3.2 Language and Tools Used

The system is developed using modern web technologies for simplicity and visualization:

- **Next.js (Frontend Framework)**
  Used to build the user interface and manage application structure.
- **TypeScript (Programming Language)**
  Ensures type safety and better code maintainability.
- **Tailwind CSS (Styling Framework)**
  Used to design a clean and responsive user interface.
- **JSON File (Data Storage)**
  Acts as a simple database to store book records.
- **OBST Algorithm (Dynamic Programming)**
  Core logic used to construct the optimal search tree.

These technologies together help in building a user-friendly and efficient system.

3.3 Implementation Description

The system works in the following steps:

Step 1: Data Input and Loading

- Book data is stored in a JSON file.
- Each record contains:
  - Book ID
  - Book Name
  - Frequency
- The data is loaded into arrays for processing.

Step 2: Sorting of Keys

- Book IDs are sorted in ascending order.
- This is necessary because OBST requires sorted keys to maintain BST properties.

Step 3: OBST Construction

- The algorithm initializes:
  - Cost matrix
  - Root matrix
- It calculates:
  - Cost of all possible subtrees
  - Best root for each subtree
- Dynamic programming is used to store intermediate results and avoid recomputation.

Step 4: Tree Construction

- Using the root matrix, the tree is built recursively.
- Each node contains:
  - Book ID
  - Left child (smaller keys)
  - Right child (larger keys)

Step 5: User Search Operation

- The user enters a Book ID.
- The system starts traversal from the root.

Step 6: Tree Traversal

- If input < current node → move left
- If input > current node → move right
- If equal → book found

During traversal:

- Each comparison is counted
- Each step is displayed to the user

Step 7: Output Display

The system shows:

- Whether the book is found or not
- Number of comparisons
- Step-by-step traversal path

This helps in analyzing efficiency.

3.4 Sample Output

Example:

Search Book ID: 20

Execution:

- Step 1: Compare with root (30) → move left
- Step 2: Compare with 10 → move right
- Step 3: Compare with 20 → found

Final Output:

- Book Found
- Comparisons = 3

This demonstrates how OBST reduces search effort by placing frequently accessed nodes near the root.

4. COMPLEXITY ANALYSIS

4.1 Time Complexity

The time complexity of the OBST algorithm is divided into two parts:

1. OBST Construction

- The algorithm uses three nested loops:
  - One for subtree length
  - One for starting index
  - One for selecting root

Thus, overall complexity:

$O(n^3)$

Where:

- n = number of keys

This is because:

- Every possible subtree is evaluated
- Every possible root is tested

2. Search Operation

- Once the tree is constructed, searching works like BST.: **O(h)**

Where:

- h = height of the tree

Since OBST is optimized:

- Height is usually smaller
- Search becomes faster on average

4.2 Space Complexity

The algorithm requires extra memory for storing intermediate results:

1. Cost Matrix

- Stores minimum cost for every subtree
- Size = n × n

👉 Complexity = **O(n²)**

---

2. Root Matrix

- Stores root index for each subtree
- Used to reconstruct tree

👉 Complexity = **O(n²)**

---

3. Tree Structure

- Stores all nodes of OBST

👉 Complexity = **O(n)**

Total Space Complexity

👉 **O(n²) (dominant term from matrices)**